Lab 4 Report: Profiling Naive vs FFT Convolution

Introduction

This lab investigates where runtime actually goes when applying convolution filters to an image using two different methods: a direct spatial (naive) convolution and an FFT-based convolution. The goal is not to make the code faster — it is to measure, observe, and explain performance behavior with real profiling evidence. The core question is: when does FFT beat naive, and why?
Convolution is a fundamental operation in image processing. For every output pixel, it computes a weighted sum of neighboring input pixels using a kernel matrix. The naive approach does this directly with nested loops. The FFT approach transforms both the image and kernel into the frequency domain, multiplies them pointwise, then transforms back — exploiting the convolution theorem to turn $O(N \cdot k^2)$ work into $O(N \log N)$ work, where N is the image size and k is the kernel side length.

Part 1 — Timing Experiments

Each command was run twice and the two average times were averaged together to reduce noise.

Experiment 1: Small-Kernel Filter (Emboss, 3×3) — FFT Should Lose

Filter	Kernel	Method	Run 1 (ms)	Run 2 (ms)	Average (ms)
emboss	3×3	naive	9.856	9.762	9.81
emboss	3×3	fft	379.90	365.95	372.92

Result: FFT is approximately 38× slower than naive for a 3×3 kernel. The hypothesis that FFT should lose here is strongly confirmed.

Experiment 2: Large Blur (Kernel-Size 51) — FFT Might Win

Filter	Kernel	Method	Run 1 (ms)	Run 2 (ms)	Average (ms)
blur	51×51	naive	2426.12	2420.78	2423.45
blur	51×51	fft	368.97	370.16	369.57

Result: At kernel-size 51, the situation completely reverses. FFT is now approximately 6.6× faster than naive.

Experiment 3: Finding the Crossover — Blur Kernel Sizes 3, 7, 15, 31, 51, 101

Filter	Kernel	Method	Average (ms)	Faster Method	FFT Speedup vs Naive
blur	3×3	naive	12.03	naive	naive 32.6× faster
blur	3×3	fft	391.44		
blur	7×7	naive	40.07	naive	naive 9.6× faster
blur	7×7	fft	385.00		
blur	15×15	naive	216.35	naive	naive 1.8× faster
blur	15×15	fft	386.54		
blur	31×31	naive	910.12	fft	FFT 2.4× faster
blur	31×31	fft	375.69		
blur	51×51	naive	2425.00	fft	FFT 6.3× faster
blur	51×51	fft	386.38		
blur	101×101	naive	9363.01	fft	FFT 23.7× faster
blur	101×101	fft	394.47		

Crossover: Naive wins at k=15 (naive 216 ms vs FFT 387 ms). FFT wins at k=31 (naive 910 ms vs FFT 376 ms). The crossover falls somewhere between k=15 and k=31.
One striking observation: FFT runtime is nearly flat across all kernel sizes, hovering between 375 ms and 394 ms regardless of whether the kernel is 3×3 or 101×101. Naive, by contrast, scales dramatically — from 12 ms at k=3 to 9363 ms at k=101. This

difference in scaling behavior is the central story of this lab and is explained in the analysis section below.

Part 2 — Profiling

Rank	% CPU (Self)	Function	Phase
1	61.04%	fft1d_inplace	FFT butterfly computation
2	22.90%	fft2d_inplace	2D FFT row/column dispatch
3	3.13%	convolve_fft	Padding, pointwise multiply, crop

The call graph confirms the dominant path: main → convolve_fft → fft2d_inplace → fft1d_inplace, with fft1d_inplace accounting for 50.85% of children time under fft2d_inplace.
~84% of FFT time is inside the FFT butterfly routines. A further ~8% is spent in memory allocation and zero-fill (page faults + clear_page_erms).

Naive Blur — Instruments Time Profiler Hotspots

Rank	% CPU (Self)	Function	Phase
1	99.58%	convolve_naive(Image const&, Kernel const&)	Triple-nested inner loop
2	0.08%	do_user_addr_fault (kernel)	Negligible
3	0.06%	rep_movs_alternative (kernel)	Negligible
—	<0.05% each	everything else (I/O, memory, init)	Negligible

Naive has essentially zero overhead. Every single CPU cycle is spent executing the innermost multiply-accumulate loop over kernel weights and image pixels.Part 3 — Analysis and Explanation

Why does FFT lose badly for emboss/edge/sharpen (3×3 kernels)?

For a 3×3 filter, the naive method only needs to perform 1024 × 768 × 9 ≈ 7 million multiply-add operations, which completes in roughly 10 ms. FFT, on the other hand, carries a large fixed overhead that does not scale with kernel size:

1. Memory allocation and zeroing. Each call to convolve_fft allocates two complex arrays of size Wp × Hp — the image padded to the next power of two. For a 1024×768 image with any kernel up to ~1000px, this is 2048×1024 = 2,097,152 elements of complex<double> (16 bytes each) per array — approximately 32 MB total. Instruments showed confirmed that 5.44% of FFT runtime is consumed by do_user_addr_fault (page faults as the OS maps these pages on first touch) and another 2.92% in clear_page_erms (zero-filling those pages). That is roughly 8% of ~370 ms = ~30 ms just for memory housekeeping, before any math begins.

2. Three FFT passes. The algorithm requires a forward FFT of the image array, a forward FFT of the kernel array, and an inverse FFT of their pointwise product. Each 2D FFT on a 2048×1024 grid is $O(Wp \cdot Hp \cdot \log_2(Wp \cdot Hp)) \approx O(2M \times 21) \approx$ 44 million butterfly operations. Three passes = ~132 million complex operations. Instruments showed shows this accounting for ~84% of runtime.

3. The overhead floor is ~370 ms regardless of kernel size. Even for the trivial 3×3 kernel, FFT must do all of the above. The naive 3×3 cost (7M ops, ~10 ms) is completely swamped by FFT's fixed startup cost of ~370 ms, making FFT ~38× slower.

The fundamental issue is that FFT's cost is driven by the image size, not the kernel size. For small kernels where naive is fast, FFT cannot possibly compete.

For blur, does FFT become competitive? At what kernel sizes?

Yes — but only once the naive method's $O(k^2)$ scaling surpasses FFT's fixed floor. The crossover occurs between k=15 and k=31:

Kernel	Naive (ms)	FFT (ms)	Winner	Ratio
15×15	216	387	naive	naive 1.8× faster
31×31	910	376	fft	FFT 2.4× faster

At k=31 naive has crossed above FFT's ~375 ms floor and loses. By k=101, naive takes 9363 ms while FFT stays at 394 ms — a 23.7× FFT advantage.
Why does FFT runtime stay flat (~370–394 ms) across all kernel sizes?

This is the most surprising result from the data and deserves a careful explanation. The FFT implementation pads the image to the next power of two large enough to accommodate a linear (non-circular) convolution: Wp = next_pow2(W + k − 1), Hp = next_pow2(H + k − 1).
For this image (1024×768), let's compute the padded size for each k:

k	W+k−1	next_pow2	H+k−1	next_pow2	Padded grid
3	1026	2048	770	1024	2048 × 1024
7	1030	2048	774	1024	2048 × 1024
15	1038	2048	782	1024	2048 × 1024
31	1054	2048	798	1024	2048 × 1024
51	1074	2048	818	1024	2048 × 1024
101	1124	2048	868	1024	2048 × 1024

Every single kernel size in this experiment maps to the identical padded grid: 2048×1024. The FFT is always operating on the same number of elements, so its runtime is constant. This is why FFT's time is flat — it's not a coincidence, it's a consequence of the image dimensions and the power-of-two alignment.
Why does naive scale quadratically with kernel size?
The inner loop in convolve_naive runs exactly k^2 times per output pixel, and there are W × H output pixels:
Total operations = $W \times H \times k^2$
Doubling k multiplies runtime by 4. The measured data confirms near-perfect quadratic scaling:

k transition	Expected ratio $(k_2/k_1)^2$	Observed ratio
k=15 → k=31	$(31/15)^2 \approx 4.27\times$	910/216 ≈ 4.21×
k=31 → k=101	$(101/31)^2 \approx 10.6\times$	9363/910 ≈ 10.3×

The match is almost exact, confirming the $O(N \cdot k^2)$ complexity of the naive implementation.

For each method, where is the CPU time going?

Naive convolution: Every cycle is in convolve_naive (99.58%). Specifically, in the innermost loop:
For k=51, this loop body executes 1024 × 768 × 2601 ≈ 2.05 billion times per convolution pass. There is no overhead at all from memory allocation, setup, or I/O — those are buried at <0.1%.

FFT convolution: Time is distributed across three distinct phases based on data:

Phase	% Time	What it does
FFT butterfly math	~84%	fft1d_inplace + fft2d_inplace computing radix-2 Cooley-Tukey FFT passes
Memory allocation	~8%	Allocating and zero-filling two 32 MB complex arrays per call
Setup / teardown	~3%	Padding image into arrays, pointwise frequency-domain multiply, cropping output
Other / OS	~5%	Miscellaneous kernel overhead

The dominant cost (84%) is the FFT computation itself — not the convolution math, not the image loading, but the butterfly passes needed to transform and invert the padded arrays.

Is performance limited by compute (math) or memory movement?

Naive — compute-bound at large kernel sizes.
For k=51, each of the 786,432 output pixels requires 2,601 multiply-add operations. The image data (1024×768 doubles ≈ 6 MB) fits comfortably within a typical L3 cache. After the first outer-loop pass warms the cache, subsequent iterations find image data already resident. The bottleneck becomes raw floating-point throughput: ~2 billion multiply-adds in ~2.4 seconds implies roughly 830 million FP ops/second — in the ballpark of what a single core can sustain with doubles and without SIMD auto-vectorization being optimal for this access pattern.
For small kernels (k=3), naive is also compute-bound, but the total work is so small that it finishes before any memory effects matter.
FFT — primarily compute-bound with a meaningful memory allocation cost.
The 84% concentration in FFT butterfly functions indicates the bottleneck is arithmetic. However, the ~8% in page fault handling and memory zeroing reveals a non-trivial secondary memory cost: every call allocates ~32 MB of new anonymous memory, which the OS must map and zero-fill on first access. This allocation overhead is a fixed cost per call that does not amortize away regardless of how much work the FFT does.
The FFT butterfly itself also has poor cache behavior compared to the naive loop. The Cooley-Tukey algorithm accesses array elements with progressively larger strides as the computation proceeds — these stride-2, stride-4, stride-N/2 access patterns generate cache misses that would not appear in a simpler sequential algorithm. This

contributes to the FFT being somewhat slower in practice than its operation count alone would suggest.

Summary

Question	Answer
When does FFT lose?	Always for small kernels ($k \leq$ ~15 on this image). FFT's fixed overhead (~370 ms) dominates naive's $O(k^2)$ cost when k is small.
When does FFT win?	For blur with $k \geq$ ~23 (crossover between k=15 and k=31). At k=101, FFT is ~24× faster.
Why is FFT runtime flat?	All kernel sizes in this experiment pad to the same 2048×1024 grid, so FFT always does the same amount of work.
Why does naive scale quadratically?	The inner loop runs k^2 times per output pixel; doubling k quadruples runtime. Confirmed empirically to within ~3%.
Where does naive spend time?	99.6% in the multiply-accumulate inner loop. Pure compute, no overhead.
Where does FFT spend time?	~84% in FFT butterfly math, ~8% in memory allocation/zeroing, ~3% in padding and pointwise multiply.
Compute or memory bound?	Naive is compute-bound. FFT is primarily compute-bound but with meaningful memory allocation overhead as a secondary cost.

The core lesson is that algorithmic complexity alone does not determine which implementation is faster — the constant factors and fixed overheads matter enormously. FFT's O(N log N) complexity is theoretically superior, but only pays off once the problem size (kernel area) is large enough to amortize its substantial fixed cost.